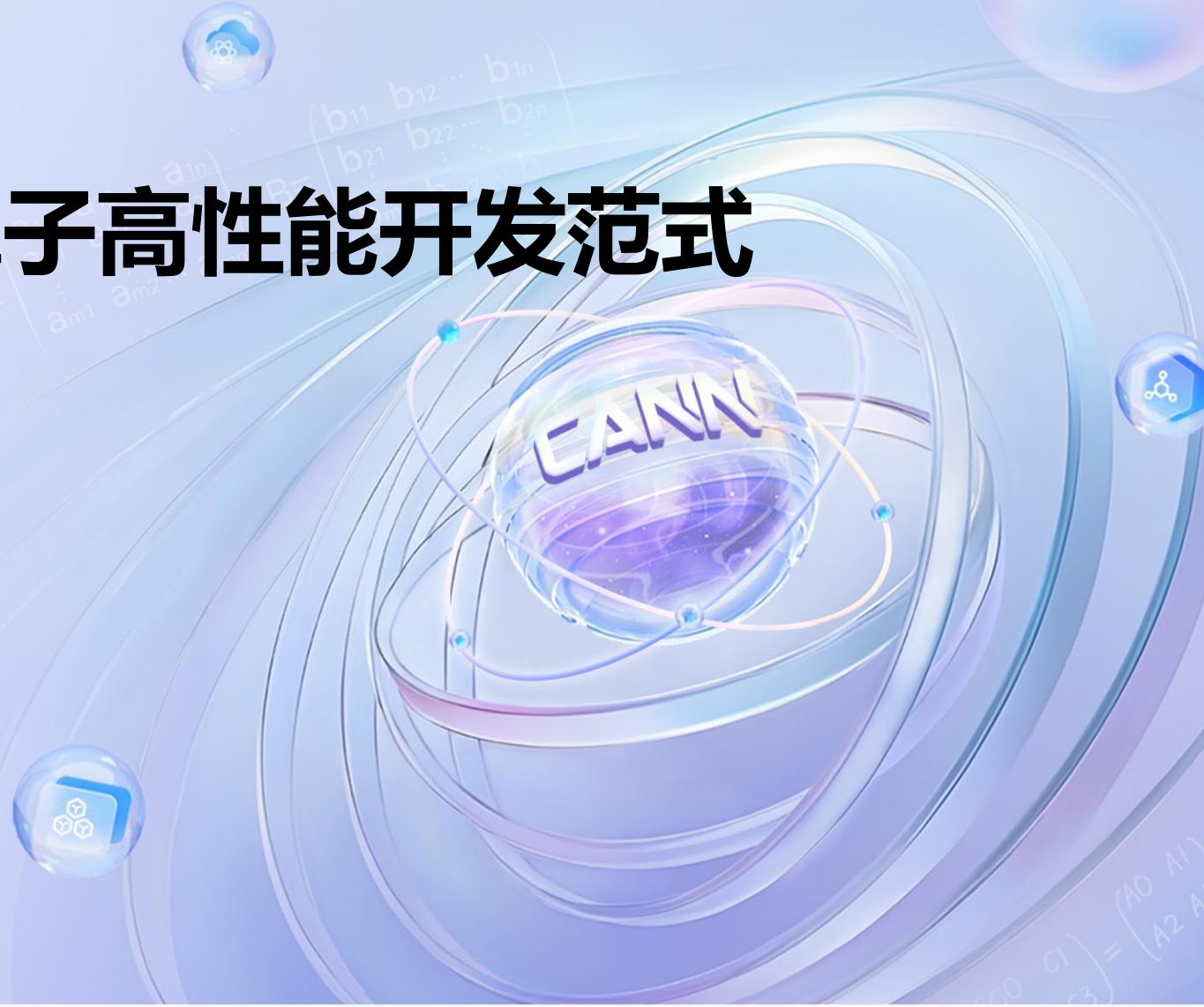
CANNBot-DSL: Agent Friendly复杂算子高性能开发范式



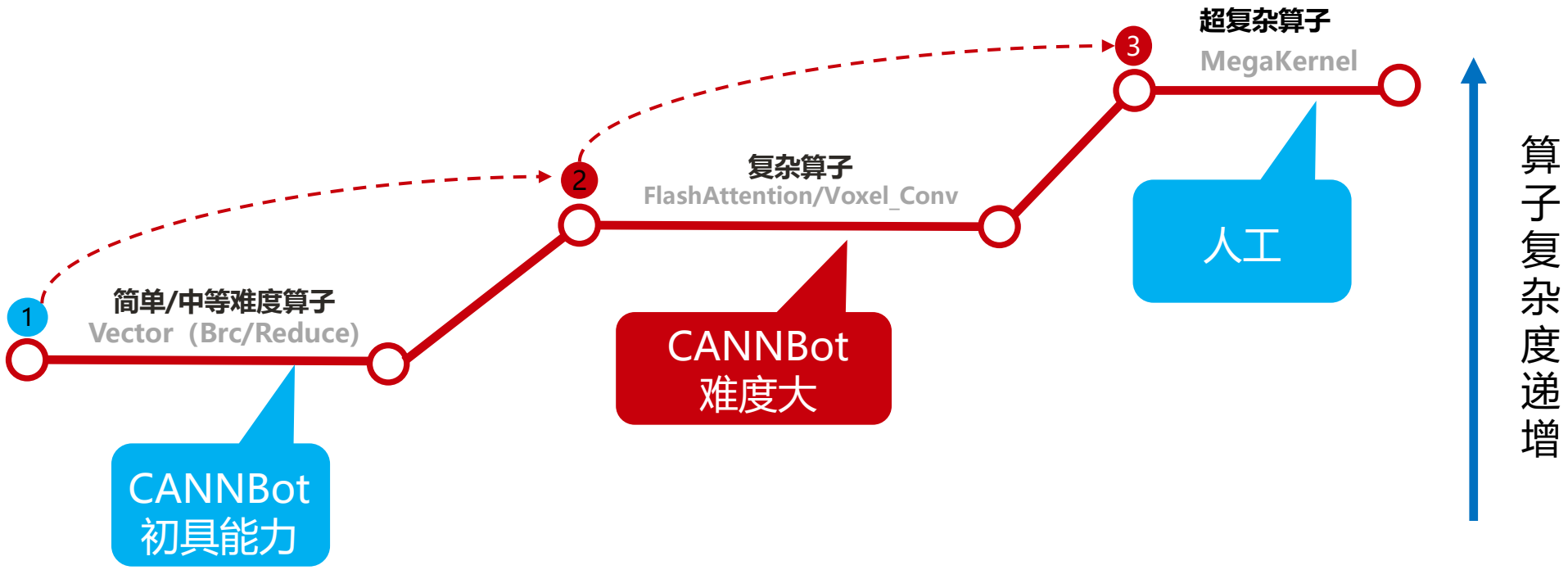
时间：2026.10.10

https://gitcode.com/cann

CANN

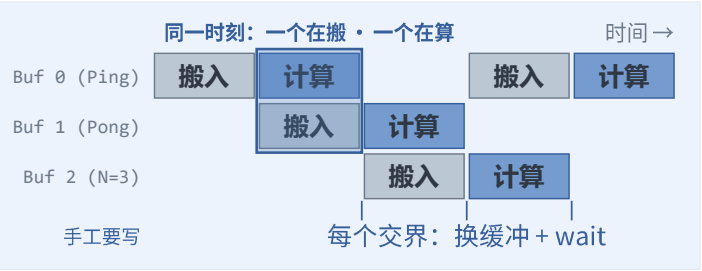
CANNBot生成算子的现状（8月份）

基于CANNBot用Ascend C能够自动生成简单&中等难度的Vector/Matmul算子，且部分可以调优到比较好高性能。 但对于复杂融合算子，受限于问题本质复杂度和LLM的能力，基于Ascend C的API还无法实现极致性能。



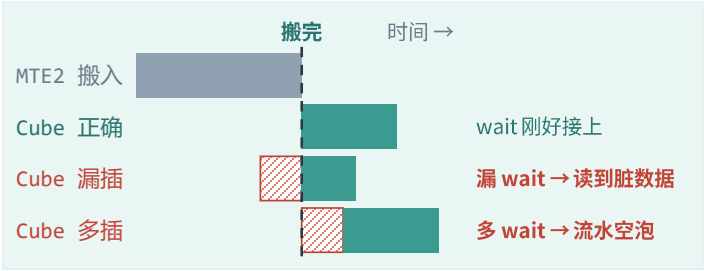
Agent生成难根因：昇腾芯片上写一个复杂融合算子会遇到如下处理逻辑

01 PingPong 写起来很复杂



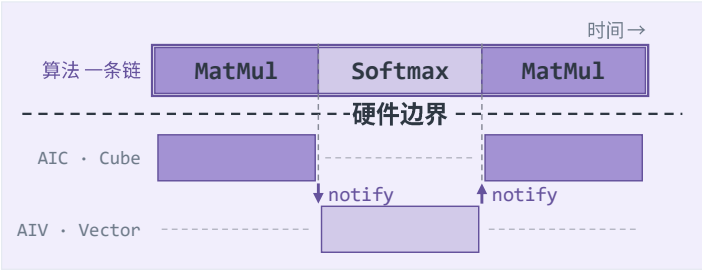
双缓冲要自己切成**偶数块**、手工交换 Ping/Pong 指针,还要决定 dma_wait() 放在循环哪一行——放错不报错,只会**性能不升反降**。PingPong 只是 **N = 2 的特例**,N 越大,指针与事件管理越复杂。

02 同步靠手工插,极易出错



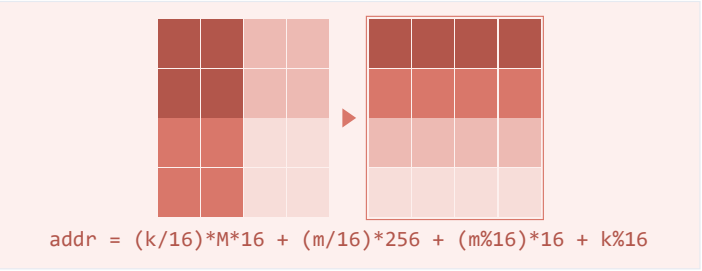
AI Core 上有六条彼此独立的指令队列,数据依赖全靠开发者手工插同步。少插一条就读到还没搬完的数据,错误随机、难复现;多插一条流水就出现空泡,性能白掉。Mix 场景还得自己把 Iterate 从同步改成异步。

03 Cube 和 Vector 代码分开写



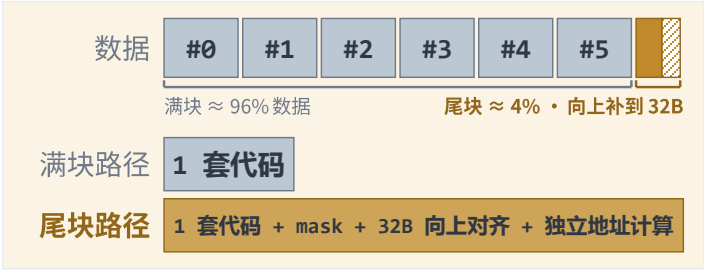
Cube 与 Vector 是两个物理核,代码要分成两份写,还要自己设计核间消息的收发时序。算法本来是一件完整的事,却被硬件边界切成两半,两侧的 tiling 和缓冲还必须严格对齐——复杂度直接翻倍。

04 私有格式处理容易出错



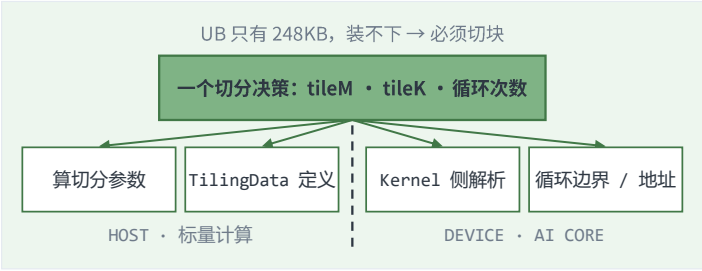
为了打满带宽和 Cube 吞吐,片上数据要用分形、NZ 这类私有排布。逻辑上的 [M,K] 和物理地址之间隔着一层非直观映射,偏移得手工推导。算错一个 stride,程序往往不崩,只是静默算错。

05 尾块需要特殊处理



真实 shape 很少正好被 tile 整除,于是每个算子都要再写一遍尾块逻辑:掩码、32B 向上对齐、单独的地址计算。分支成倍增加,而尾块通常只占总数据量的几个百分点——收益极低,却是 bug 高发区。

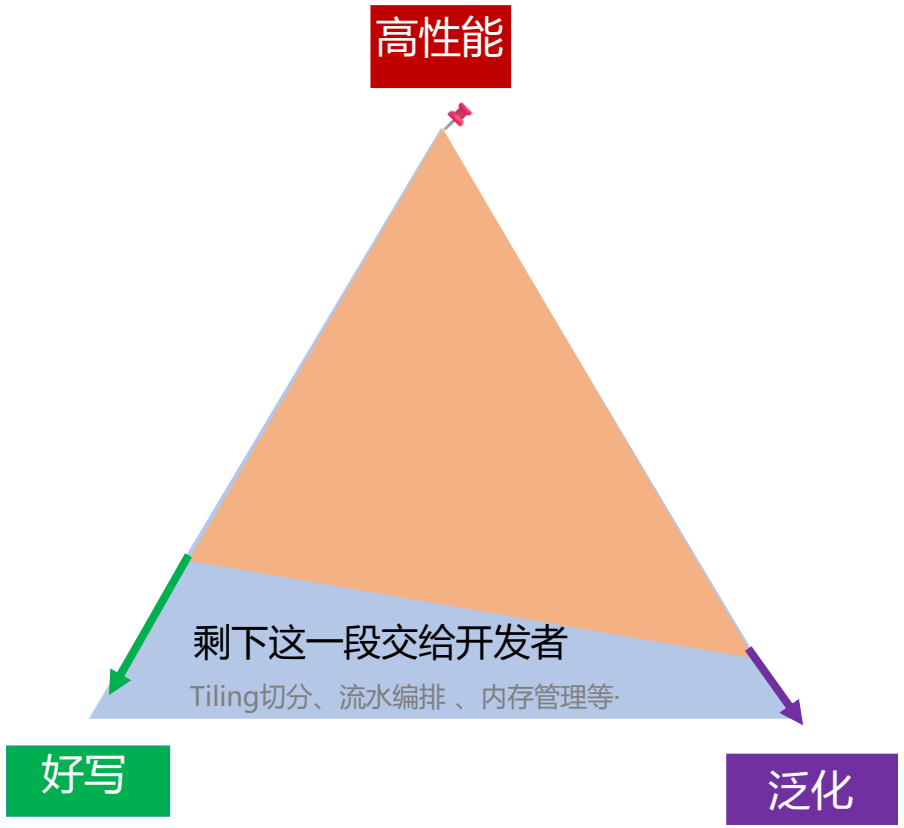
06 Tiling 链路横跨 Host 与 Kernel



UB 只有 248KB,装不下完整的输入输出,必须切块。而切分参数这一个决定,要在 Host 与 Kernel 两侧四处分别写一遍——改一处就得同步改四处,动态 shape 还要在运行期重算。

设计理念1：固定住高性能这个角，尽可能的易写和通用

性能、效率、通用三者不可兼得，这是编译器领域的老问题，不是昇腾独有。CANNBot-DSL 的取舍是：固定**高性能**这端，再从这一角出发,把 **好写** 和 **泛化** 尽量往下伸



高性能	所有的建模和抽象的前提是在编译期完成相关的计算和推导，在运行期是性能无损的。例如同步的管理、多Buffer的管理、CV无感知编码等交给DSL框架。
好写	由于做了较好的建模和抽象，算在编码复杂度下降了较多，但并不意味着可以只写算法，对于Tiling切分、流水编排、内存管理等性能相关项，依然由开发者控制。
泛化	基于Ascend C编程体系，为确保和Ascend C泛化能力一致，支持动态Shape场景，也支持动态Tiling场景。支持二进制和JIT方式接入。

设计理念2：开发者的代码尽可能的描述算法逻辑

意图描述层

开发者 + Agent，只声明做什么，算法怎么算、数据怎么流

算法本身

MatMul → Softmax → MatMul 一条链

Channel(..., depth=N)

要几级流水,就声明几

.produce() / .consume()

只说谁生产、谁消费

以下由框架负责(pass)

多级流水

~~swap(ping,pong) · dma_wait()~~

由 depth 推导展开、轮转与尾块

队列同步

~~set_flag / wait_flag~~

由数据依赖推导,不漏不多

核间边界

~~两份 Kernel + 消息时序~~

算法只写**一条链**,切分交给框架

私有格式

~~addr = (k/16)*M*16 + ...~~

由访存抽象推导分形 / NZ 地址

尾块处理

~~mask + 32B 对齐 + 独立地址~~

边界分支由框架补齐

Tiling 链路

~~TilingData · 改一处动四处~~

切分声明一次,Host / Kernel 贯通

Agent Friendly：写昇腾算子更接近于算法表达

能被推导的,就不该被手写

流水轮转、同步位置、地址偏移、尾块分支,都被结构声明唯一确定——凡是能推导出来的,就不该再占用人的注意力。

抽象零开销

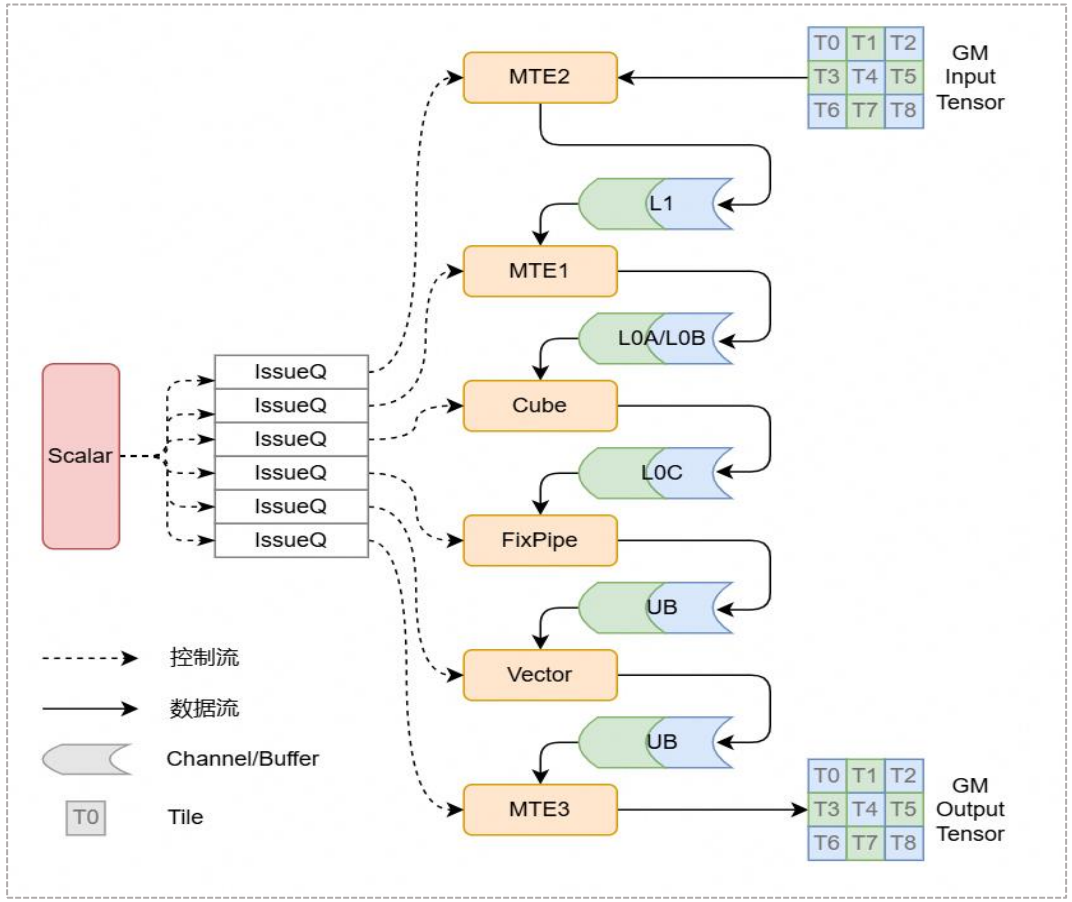
分形、NZ 本来就是为性能而存在。抽象不是消灭它们,而是封装它们——少算地址、少出错,但不为易用性让出性能。

基于成熟的Ascend C体系构建

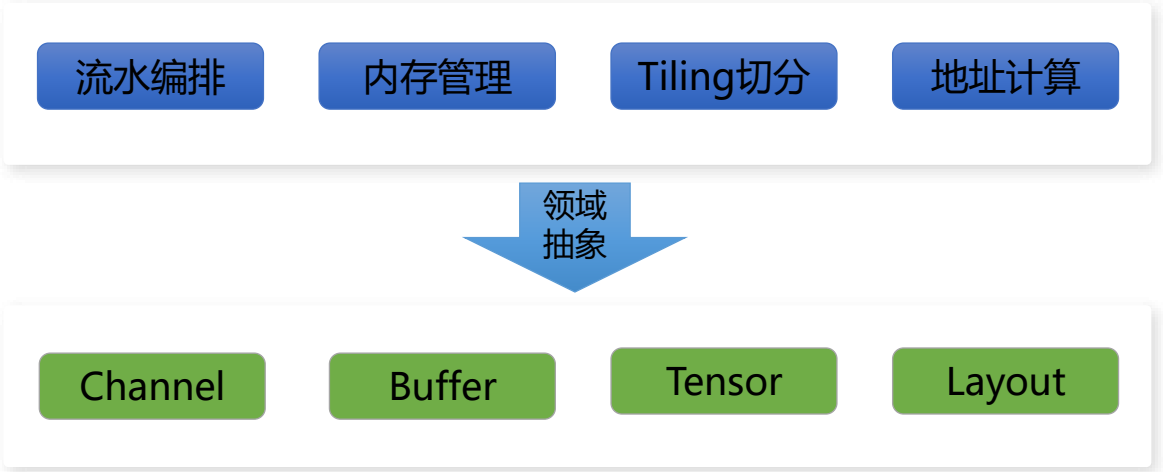
Python DSL + MLIR ,建在成熟Ascend C体系之上,
可以复用底层的Bisheng编译器优化能力。

设计理念3：亲和昇腾昇腾芯片的前端表达，性能无损

1. **建模抽象：**将算子复杂的异步并发编程范式，通过亲和NPU芯片进行建模，抽象为简单的串行的语义表达（实际执行为异步并发）(A)
   将算子在计算过程中的复杂的计算逻辑进行建模，抽象为可复用、表达完备的软件组件，消除了冗余复杂的计算过程 (B)
2. **建模抽象原则：**为达到极致性能，建模抽象进行的非常克制：只对性能无损的流程进行建模抽象，性能有损的，交给开发者。



将芯片异步并行计算抽象为面向软件串行计算



基于协程模型和领域抽象构建出昇腾亲和的编程范式：

- 抽象Channel作为异步操作数据流的管道，内部自动处理异步执行单元间的同步插入以及Buffer的自动切换
- 算子Kernel编程操作的主体从裸指针转变为Tensor；
- 基于Layout表达数据Tile块的切分以及Tile块在核间/核内的调度；

将串行复杂的计算逻辑抽象为软件组件简化计算表达

设计结果：算子的代码更接近CPU上的表达

Attention的例子：Attention 的 QKT → Softmax → PV，Cube 和 Vector 都要用上、多Buffer，私有格式、跨核搬运、尾块等都需要开发者关注。但是通过CANNBot-DSL的表达来实现，这些细节都可以由DSL的框架（编译pass）完成。

CPU / NumPy

Attention的算法如下，没有硬件相关的信息

```
s = q @ k.T

m = s.max(axis=-1)
e = exp(s - m)
p = e / e.sum(axis=-1)

o = p @ v
```

5 行,描述的是算法

前面提到的那六件事，全由框架完成了

~~01 ping / pong 轮转~~

~~02 set_flag / wait_flag~~

~~03 Cube / Vector 拆核~~

~~04 分形 / NZ 地址~~

~~05 尾块 + 32B 对齐~~

~~06 TilingData 跨端~~

≈

算法那五行非常接近

CANNBot-DSL

算子开发这描述数据往哪搬，然后交由编译器算地址、时序与同步等

```
nd2nz  = make_copy_engine(format_transform="nd2nz")   私有格式声明一次
fixpipe = make_copy_engine(dtype=fp32)                L0C 搬出
s = Channel(MemLoc.UB, (BM, N), fp32, depth=2)        tile / depth 声明一次

for mi in range(ceil_div(S_q, BM)):                   尾块编译器补
    mem_copy(q_l1, q_gm[mi], engine=nd2nz)            GM → L1
    mem_copy(k_l1, k_gm,     engine=nd2nz)            分形地址不用手算
    mem_copy(l0a, q_l1); mem_copy(l0b, k_l1)          L1 → L0A / L0B
    matmul(l0c, l0a, l0b, init=True)                  Cube:S = Q·Kᵀ，同左
    mem_copy(s, l0c, engine=fixpipe)                  L0C → UB · 跨核交棒

    m = s.max(axis=-1)                                Vector · 同左
    e = exp(s - m)                                    同左
    p = e / e.sum(axis=-1)                            同左
    mem_copy(p_l1, p); mem_copy(l0a, p_l1)            P 回 Cube
    mem_copy(l0b, v_l1, transpose=True)               转置也是声明
    matmul(l0c, l0a, l0b, init=True)                  Cube:O = P·V，同左
    mem_copy(o_gm[mi], l0c, engine=fixpipe)           UB → GM
```

多出来的 11 行,主要是声明、Tiling循环、和访存

CANNBot-DSL代码示例

同一份 Python 算子：Device 实现、Host 计算与 torch 调用

Device 计算

```python
@kernel
def matmul_kernel(y: Tensor, a: Tensor, b: Tensor):
    l1_a = Channel(MemLoc.L1, (TM, KL1), dtypes.float16, depth=2)
    l1_b = Channel(MemLoc.L1, (TN, KL1), dtypes.float16, depth=2)
    l0a = Channel(MemLoc.L0A, (TM, TK), dtypes.float16, depth=2)
    l0b = Channel(MemLoc.L0B, (TN, TK), dtypes.float16, depth=2)
    l0c = Channel(MemLoc.L0C, (TM, TN), dtypes.float32, depth=2)
    cv_ub = Channel(MemLoc.UB, (TM // 2, TN), dtypes.float32, depth=2, kind=ChannelKind.CrossCore)
    out_ub = Channel(MemLoc.UB, (TM // 2, TN), dtypes.float32, depth=2)
    nd2nz = make_copy_engine(format_transform="nd2nz")
    split_m = make_copy_engine(split_axis=0)
    for m in range(get_block_idx() // 8, a.shape[0] // TM, 4):
        for n in range(get_block_idx() % 8, b.shape[0] // TN, 8):
            l0c_acc = l0c.produce()
            y_tile = tile_slice(y, (TM, TN), (m, n))
            for k1 in range(a.shape[1] // KL1):
                mem_copy(l1_a.produce(), tile_slice(a, (TM, KL1), (m, k1)), engine=nd2nz)
                mem_copy(l1_b.produce(), tile_slice(b, (TN, KL1), (n, k1)), engine=nd2nz)
                a_l1, b_l1 = l1_a.consume(), l1_b.consume()
                for k0 in range(KL1 // TK):
                    mem_copy(l0a.produce(), tile_slice(a_l1, (TM, TK), (0, k0)))
                    mem_copy(l0b.produce(), tile_slice(b_l1, (TN, TK), (0, k0)))
                    a_l0, b_l0 = l0a.consume(), l0b.consume()
                    global_k = k1 * (KL1 // TK) + k0
                    matmul(l0c_acc, a_l0, b_l0, init=(global_k == 0))
            mem_copy(cv_ub.produce(), l0c.consume(), engine=split_m)
            cv_tile, out_tile = cv_ub.consume(), out_ub.produce()
            with vf(mode="simd"):
                for row in range(TM // 2):
                    for col in range(0, TN, 64):
                        mask, _ = rr.update_mask(TN - col, elem_bits=32)
                        value = rr.vload(cv_tile, row * cv_tile.physical_stride[0] + col)
                        scaled = rr.vmuls(value, 0.5, mask=mask)
                        rr.vstore(out_tile, row * out_tile.physical_stride[0] + col, scaled, mask)
            mem_copy(y_tile, out_ub.consume(), engine=split_m, part_id=get_subblock_id())
```

Host 计算

```python
@host
def matmul_host(y: Tensor, a: Tensor, b: Tensor):
    matmul_kernel[32](y, a, b)
```

框架调用

```python
a = torch.randn(M, K, dtype=torch.float16)
b = torch.randn(N, K, dtype=torch.float16)
y = torch.zeros(M, N, dtype=torch.float32)
matmul_host(y, a, b)
```

易用性提升

Python 开发闭环
原生语法、编译调试、验证调用

Tensor / Tile 表达
数据计算流、切分与搬运

Cube / Vector 代码自动划分
无需显式标注，编译器自动判定归属

同步与Buffer自动管理
同步自动插入、Buffer循环复用

极致性能表达

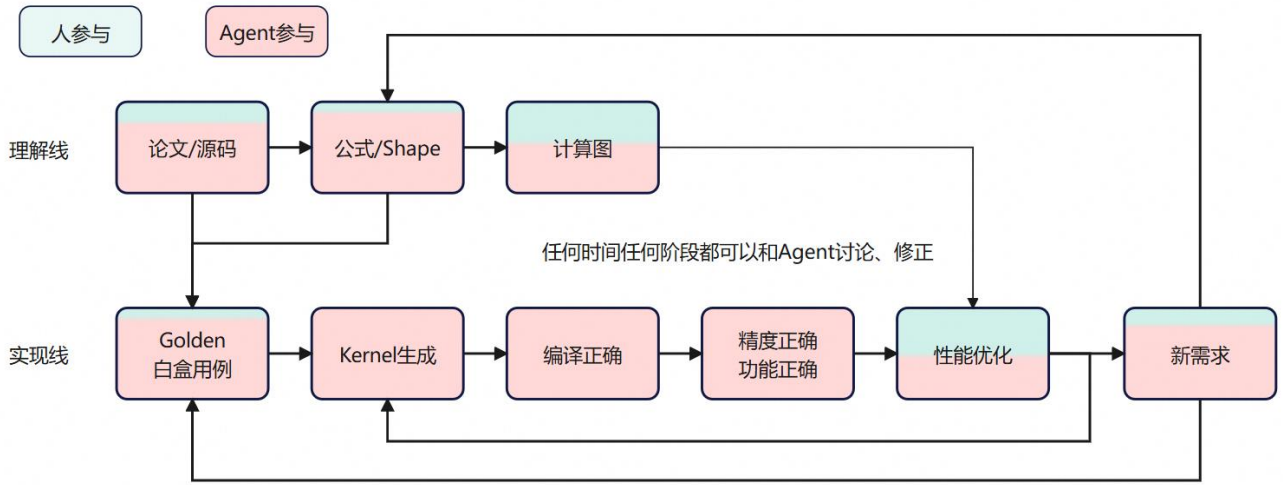
分块与调度可控
Tiling切分、Tile块调度

高效指令可控
计算指令、搬运指令

Bank 优化可控
L1/UB Bank冲突解决

流水编排可控
计算搬运协同

开发者和CANNBot都要干什么



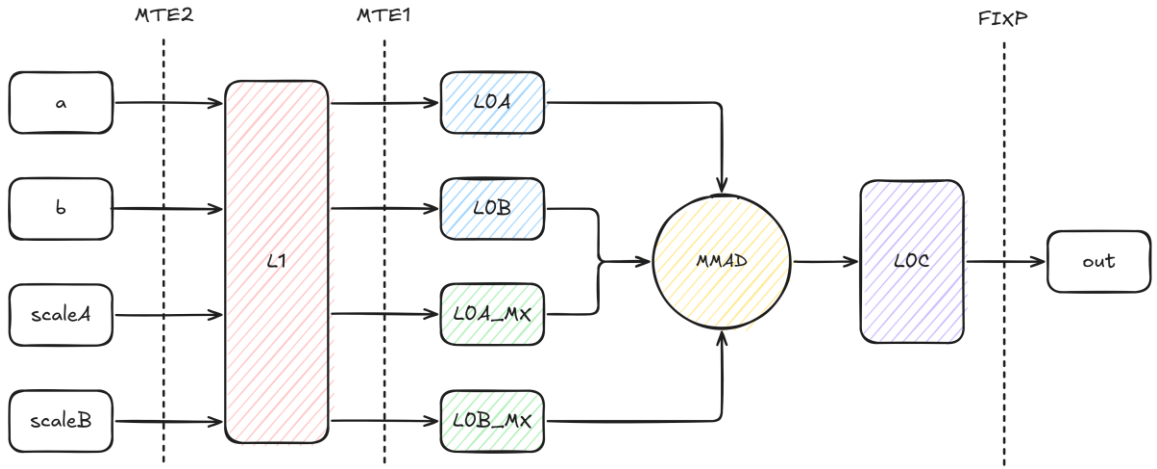
角色	主要职责
开发者 目标定义、方案审核 最终验收	**1.需求规格化与边界锁定**：明确算子功能、规格、特性等；同时定义精度容差、性能基线和验收标准。产出：《算子需求与验收标准》。 **2.掌握计算流并审核 Agent 方案、结果与代码**：吃透算子的数学语义、数据流等；在此基础上审核 Agent 给出的方案、结果和代码，判断正确性、决定优化路线。产出：审核意见、决策记录、优化路线。 **3.拆解可验证任务并设置门禁与最终验收**：整体目标拆成阶段：接口/骨架、golden、白盒测试、精度测试、性能 profiling、回归测试等；每阶段定义交付物、通过标准和人工门禁，最终做验收。产出：阶段任务表、风险记录、验收结论。
Agent 调研分析、实现验证 协同记录	**1.参考调研与静态审查**：阅读论文、参考实现，梳理算子计算流、shape 推导、边界条件和依赖；输出方案对比、风险清单、初步优化候选。产出： 调研报告、计算流说明、优化列表。 **2.kernel实现与白盒精度验证**：按阶段实现或修改代码，构造 golden，完成白盒测试、边界测试、精度测试和失败案例复现；保证实验可复现。产出： 代码/diff、golden、测试报告、问题清单。 **3. 性能实验、讨论与知识沉淀**：建立 baseline，做 profiling、瓶颈定位、A/B 优化和回归验证；协助开发者理解算子，维护任务记录、实验日志和决策记录。产出： 性能报告、实验记录、讨论结论、任务记录。

基于CANNBot-DSL的MX 量化Matmul实践

$$c_{i,j} = \sum_{g=0}^{\mathrm{ceil}(K/G)-1} \left(\mathrm{scaleA}_{i,g} \cdot \mathrm{scaleB}_{g,j} \cdot \sum_{k'=0}^{G-1} \left(a_{i,gG+k'} \cdot b_{gG+k',j} \right) \right)$$

每 32 个 K 元素共享 Scale，接口按两个 Scale 一组存储

变量名	描述	Dtype	Shape
a	输入左矩阵	float8_e4m3fn	(m, k)
b	输入右矩阵	float8_e4m3fn	(n, k)
scaleA	左矩阵量化参数	float8_e8m0	(m, ceil(k/64), 2)
scaleB	右矩阵量化参数	float8_e8m0	(n, ceil(k/64), 2)
c	输出矩阵	bfloat16	(m, n)




```
# GM → L1: 格式转换
copy_a_gm_to_l1 = make_copy_engine(
    format_transform="nd2nz")
mem_copy(l1_a.produce(), a_gm_tile,
         engine=copy_a_gm_to_l1)

# L1 → L0: 数据 + MX Scale
mem_copy(l0a.produce(), a_l1_tile,
         mx_scale=scale_a_l1_tile)

# Cube 计算与 FIXPIPE 写回
matmul(l0c_slot,
       l0a.consume(), l0b.consume(), ...)
mem_copy(out_tile, l0c_slot,
         engine=fp, unit_flag=3)
```

显式表达搬运与计算策略

三层组织：执行边界明确，资源显式传递

装饰器区分入口与内联辅助，Channel 声明资源，调度与计算沿清晰接口衔接



@kernel：创建资源，组织多核执行

```
@kernel
def qbmm_kernel(self, out_gm, a_gm, b_gm,
                scale_a_gm, scale_b_gm, bias_gm):
    l1_a = Channel(MemLoc.L1, (base_m, k_l1), a_dtype,
                   depth=l1_buffers, data_format="nz")
    # ... 其余资源与搬运引擎省略
    block_idx = get_block_idx()
    round_count = self.block_scheduler.get_round_count(block_idx)
    for round_idx in range(round_count):
        m_idx, n_idx, tail_split, sub_m_idx, sub_n_idx = (
            self.block_scheduler.get_tile_idx(
                block_idx, round_idx, round_count))
        self._compute_output_tile(*tile_ctx, m_idx, n_idx,
                                  bias_gm, tail_split,
                                  sub_m_idx, sub_n_idx)
```

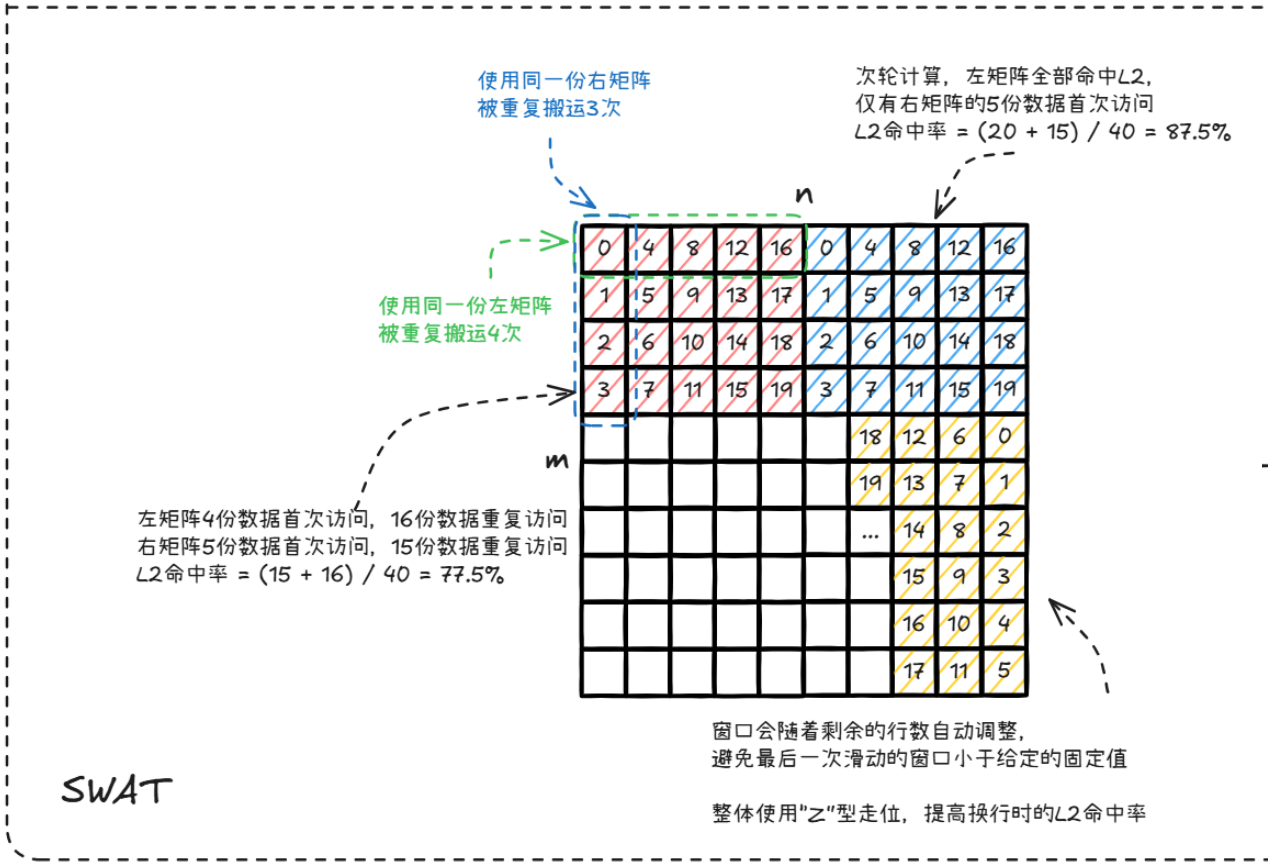
@host：编译与执行入口

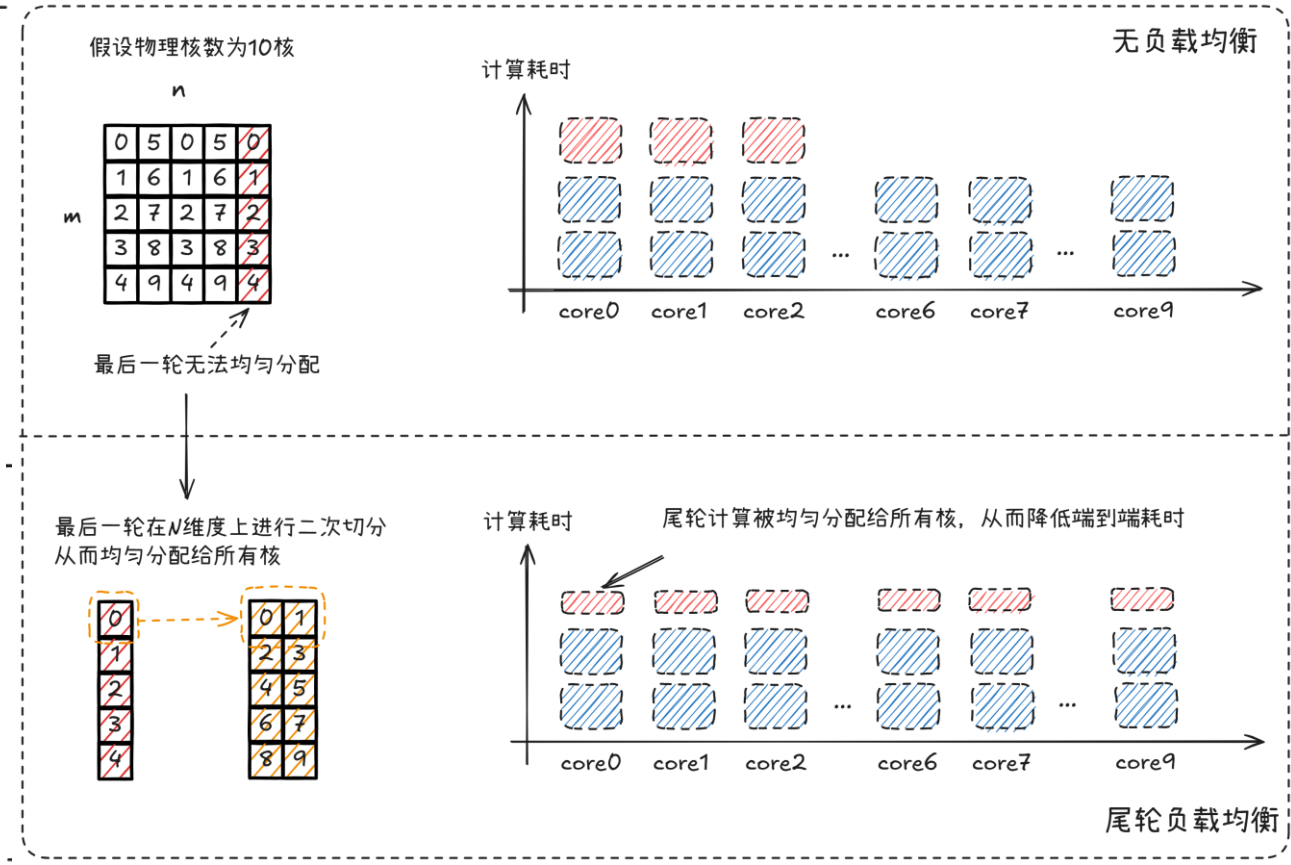
```
@host
def run(self, out_gm, a_gm, b_gm,
        a_scale_gm, b_scale_gm,
        bias_gm):
    self.qbmm_kernel[
        self.used_core_num](
        out_gm, a_gm, b_gm, ...)
```

@jit：调用处内联核内计算

```
@jit
def _compute_output_tile(self, out_gm, a_gm, b_gm,
    # ... 其余资源与索引参数省略
):
    # 解析范围，切片与搬运
    # MMAD 累加，FIXPIPE 写回
    ...
```

Block Schedule：每个核处理哪块Tile





自适应滑动窗口

改变多核访问局部性，增加 A/B 复用机会

尾轮切分

让空闲核参与尾轮子块计算

Block Schedule：每个核处理哪块Tile

走位与切分直接可分析、可修改，调度结果通过 Tensor 切片衔接搬运

get_tile_idx：走位与切分显式可控

```
w, nt = self.main_window, self.n_tiles  # 展示用别名
logical_idx = round_idx * self.used_core_num + block_idx
sub_m_idx, sub_n_idx = 0, 0
# tail_split 已按开关、最后一轮与参与核条件计算
if tail_split:
    logical_idx = (
        round_idx * self.used_core_num
        + block_idx // self.total_tail_tile)
    split_idx = block_idx % self.total_tail_tile
    sub_m_idx = split_idx % self.m_tail_tile
    sub_n_idx = split_idx // self.m_tail_tile
if logical_idx < self.total_main_tiles:
    row_idx = logical_idx // (nt * w)
    m_idx = row_idx * w + logical_idx % w
    n_forward = (logical_idx // w) % nt
else:
    tail_idx = logical_idx - self.total_main_tiles
    row_idx = self.main_row
    m_idx = self.main_row * w + tail_idx % self.tail_window
    n_forward = (tail_idx // self.tail_window) % nt
reverse = row_idx % 2
n_idx = n_forward + reverse * ((nt - 1) - 2 * n_forward)
return (m_idx, n_idx, tail_split,
        sub_m_idx, sub_n_idx)
```

get_block：范围直接驱动搬运

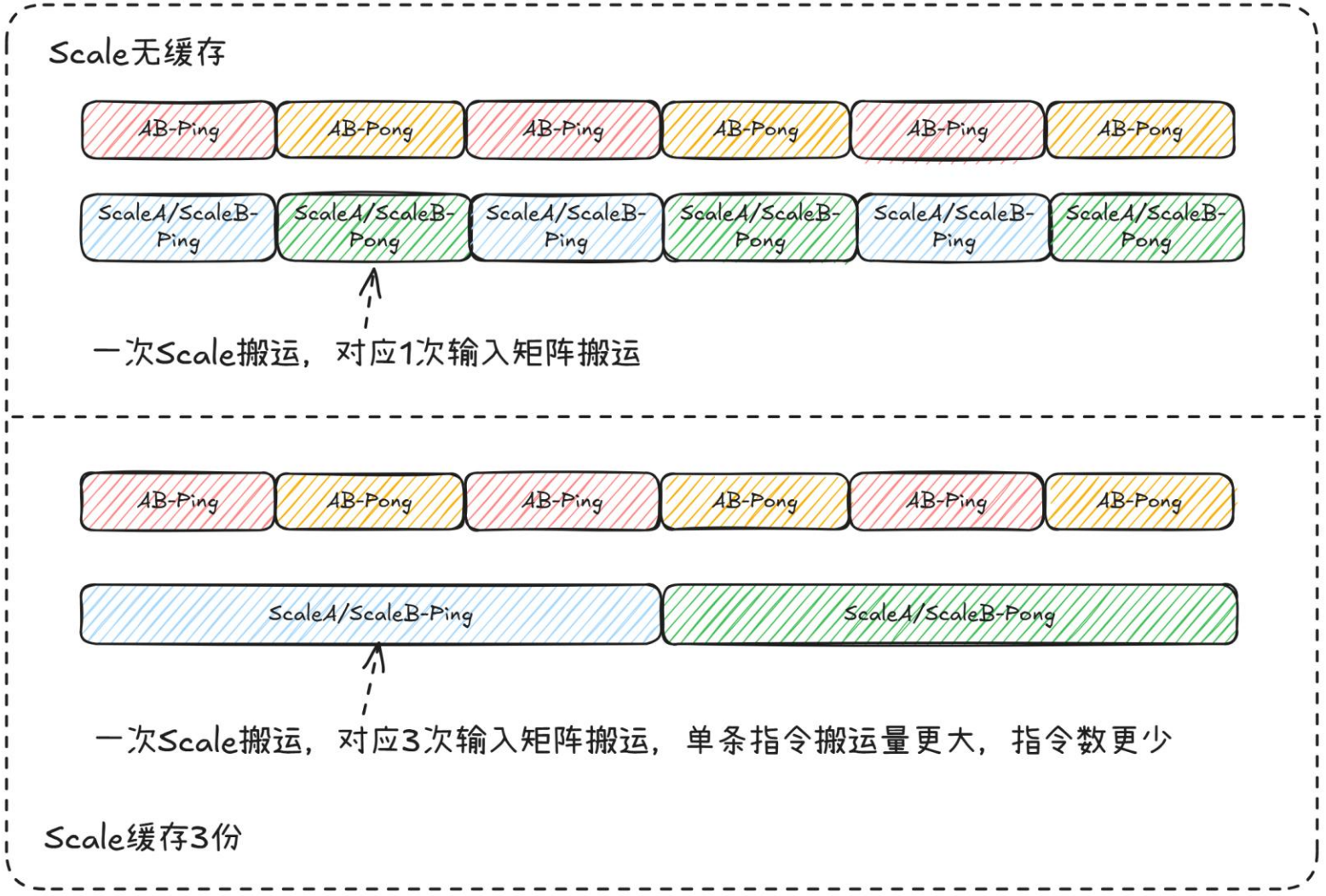
```
m_offset, n_offset = 0, 0
if tail_split:
    sub_m = ceil_div(current_m, self.m_tail_tile)
    sub_n = ceil_div(current_n, self.n_tail_tile)
    m_offset = sub_m_idx * sub_m
    n_offset = sub_n_idx * sub_n
    current_m = min(current_m - m_offset, sub_m)
    current_n = min(current_n - n_offset, sub_n)
# ... 原 Tile 起点已计算，返回子块起点与尺寸
return (m_start + m_offset, n_start + n_offset,
        current_m, current_n)

# _compute_output_tile 内：消费返回值
m_start, n_start, current_m, current_n = (
    self.block_scheduler.get_block(
        m_idx, n_idx, tail_split, sub_m_idx, sub_n_idx))
m_end = m_start + current_m
n_end = n_start + current_n
a_gm_tile = a_gm[m_start:m_end, k_start:k_end]
mem_copy(l1_a.produce(), a_gm_tile,
         engine=copy_a_gm_to_l1)
b_gm_tile = b_gm[n_start:n_end, k_start:k_end]
mem_copy(l1_b.produce(), b_gm_tile,
         engine=copy_b_gm_to_l1)
```

策略有明确修改位置，数据访问用[]运算符灵活表达，核内复用同一接口

Scale 缓存与三个 K 分块粒度

调优有抓手：Scale 复用范围、数据搬运窗口与计算粒度分别表达



示意参数

base_k = 128
k_l1 = 256
scale_k_l1 = 768

一个 Scale 窗口
覆盖 3 个数据窗口
每窗口包含 2 个 L0 子块

代码落点

Scale 外层循环
数据 L1 窗口循环
baseK 子块循环

Block Mmad：循环与引用显式表达复用

Scale 切块、L1切块与 L0 切块分别组织，mx_scale 直接关联量化参数

复用范围：一次 Scale 加载覆盖多个数据窗口

```
# 仅展示 A; B 路径省略
for scale_l1_idx in range(self.scale_l1_tiles):
    # ... 建立当前 Scale 的 GM 切片
    mem_copy(l1_scale_a.produce(), scale_a_gm_tile,
             engine=copy_scale_a_gm_to_l1, l2_cache_ctl=1)
    scale_a_l1_rd = l1_scale_a.consume()
    # ... 按有效尾窗计算 current_scale_k_l1_factor
    for k_l1_idx_in_scale in range(current_scale_k_l1_factor):
        # ... 计算当前 k_start / k_end
        a_gm_tile = a_gm[m_start:m_end, k_start:k_end]
        mem_copy(l1_a.produce(), a_gm_tile,
                 engine=copy_a_gm_to_l1,
                 l2_cache_ctl=self.l2_cache_ctl_a)
        a_l1_rd = l1_a.consume()
        for k_l0_idx in range(current_step_k):
            # 复用上层引用，执行右侧 L1→L0 与 MMAD
            ...
```

k_l0_idx 循环内：切块并关联 Scale

```
# full_load=False
a_k_l0_idx = k_l0_idx
scale_a_k_l0_idx = (
    k_l1_idx_in_scale * self.step_k + k_l0_idx)
a_l1_tile = tile_slice(a_l1_rd,
    (self.base_m, self.base_k), (0, a_k_l0_idx))
scale_a_l1_tile = tile_slice(scale_a_l1_rd,
    (self.base_m, self.scale_k_l0_len),
    (0, scale_a_k_l0_idx))
mem_copy(l0a.produce(), a_l1_tile,
         mx_scale=scale_a_l1_tile)
# ... B 数据与 Scale 加载，累加参数省略
matmul(l0c_slot,
       l0a.consume(), l0b.consume(), ...)
```

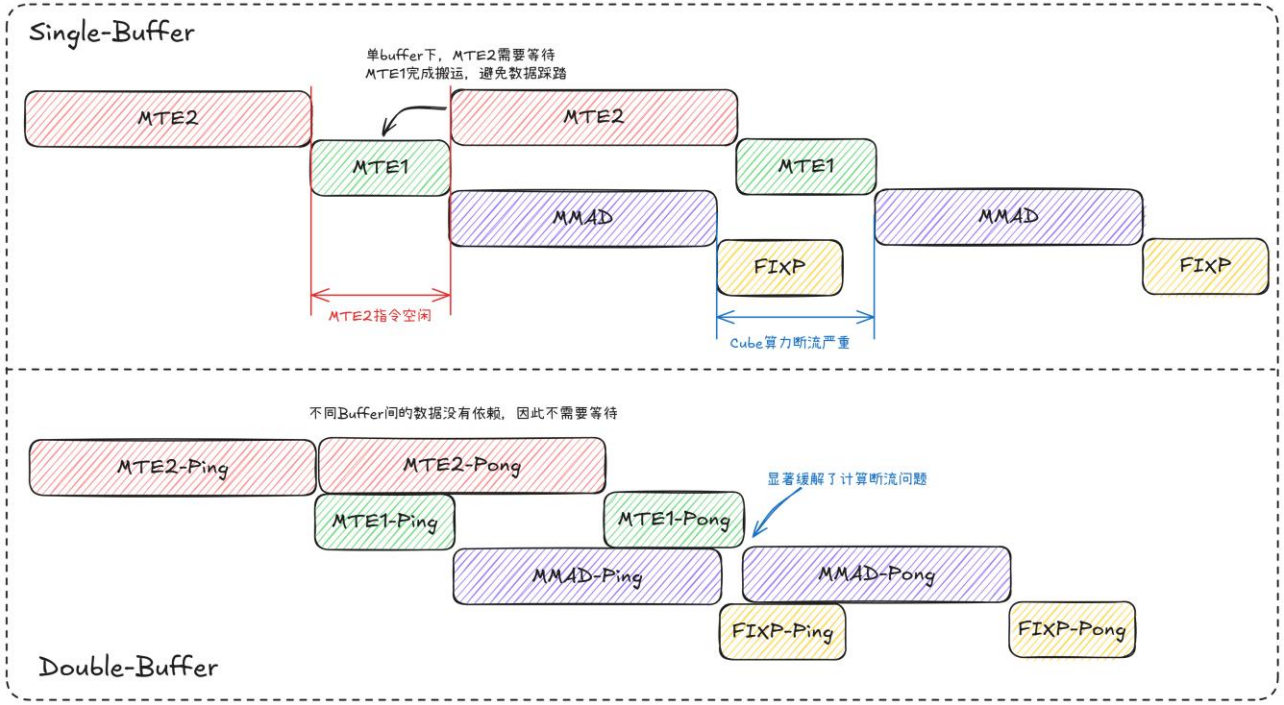
复用由循环层次与 Tensor 引用表达

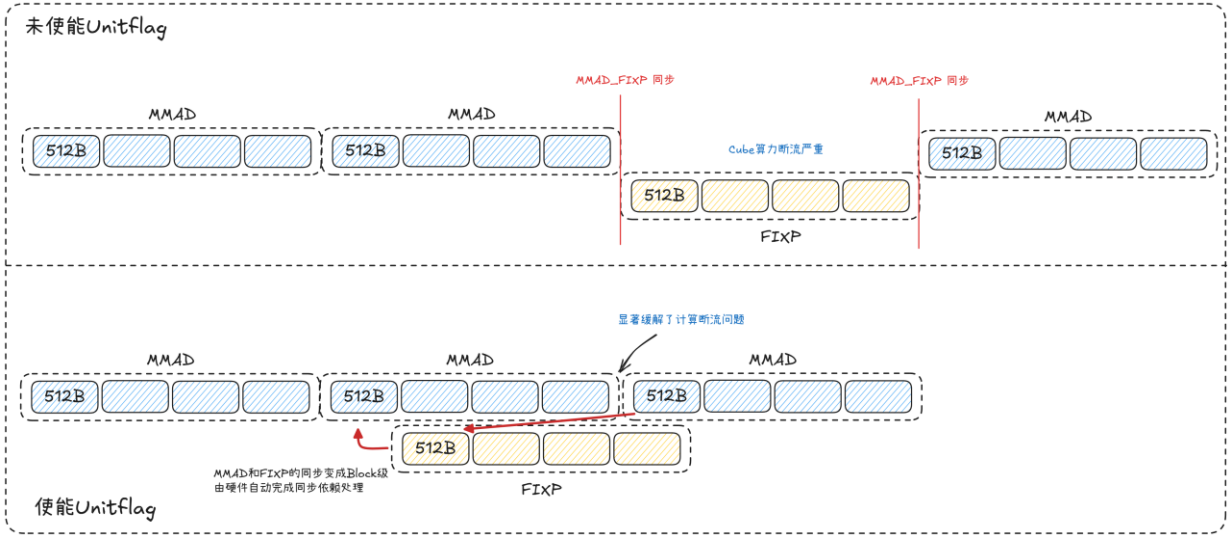
Scale 在外层加载并选槽一次
多个数据窗口复用同一引用
L0 加载直接关联当前 Scale 子块

一次生产多次消费，全程基于Tensor切片衔接搬运，分块粒度有明确调优位置

流水优化：缓冲轮换与硬件交接

性能可分析：沿资源生命周期理解重叠机会与依赖关系





多缓冲：轮次使用不同槽

Channel depth 表达存储槽数
L0C 在输出 Tile 之间轮换，Tile 内保持同槽

UnitFlag：MMAD 与 FIXPIPE 交接

中间 MMAD：2，最终 MMAD：3
FIXPIPE 写回同样使用 unit_flag=3

Block Mmad：多级流水与累加行为显式可控

多级流水与累加行为显式可控，常规资源同步由编译器承接

```
out_tile = out_gm[m_start:m_end, n_start:n_end]
l0c_slot = l0c.produce()       # 每个输出 Tile 只选一次

# ... Scale / k_l1 两层窗口循环
for k_l0_idx in range(current_step_k):
    global_k_l0_idx = global_k_l1_idx * self.step_k + k_l0_idx
    # ... 搬入 L0A / L0B，并携带对应 Scale
    is_final_acc = global_k_l0_idx + 1 == self.k_l0_tiles
    matmul(
        l0c_slot,
        l0a.consume(), l0b.consume(),
        init=(global_k_l0_idx == 0),
        bias=bias,
        unit_flag=3 if is_final_acc else 2,
    )
# 全部 K 累加之后，FIXPIPE 写回这个 Tile
mem_copy(out_tile, l0c_slot, engine=fp,
         unit_flag=3, l2_cache_ctl=1)
```

生命周期直接可读

Tile 开始时选槽
跨所有 K 子块保持
写回同一个槽

累加行为显式可控

首次初始化，随后累加

硬件交接有明确落点

中间 MMAD：2
最终 MMAD / FIXPIPE：3
配合硬件交接

Channel：资源访问与自动同步

相同执行步骤：L0 双缓冲搬运、MMAD 计算、槽位再次复用

CANNBot-DSL：表达数据访问

L0A / L0B 使用 Channel(depth=2)

```
# 1. 搬运到当前写入槽
mem_copy(l0a.produce(), a_l1_tile,
         mx_scale=scale_a_l1_tile)
mem_copy(l0b.produce(), b_l1_tile,
         mx_scale=scale_b_l1_tile,
         transpose=True)
```

```
# 2. 从当前读取槽执行 MMAD
matmul(l0c_slot,
       l0a.consume(), l0b.consume(), ...)
```

```
# 3. 槽位轮换与访问同步由DSL框架承接
```

抽象前：显式管理槽位与同步事件

L0A / L0B 使用双缓冲，显式配对硬件同步事件

```
// 1. 等待当前槽可复用，再搬运
auto id = l0PingPong_ & 1;
auto offset = halfL0Size * id;
auto flag = static_cast<uint16_t>(
    id + M_MTE1_FLAG_0);
WaitFlag<HardEvent::M_MTE1>(flag);
auto a = CopyAToL0(..., offset);
auto b = CopyBToL0(..., offset);
```

```
// 2. 等待搬运完成，再执行 MMAD
SetFlag<HardEvent::MTE1_M>(id);
WaitFlag<HardEvent::MTE1_M>(id);
Mmad(..., a, b, ...);
```

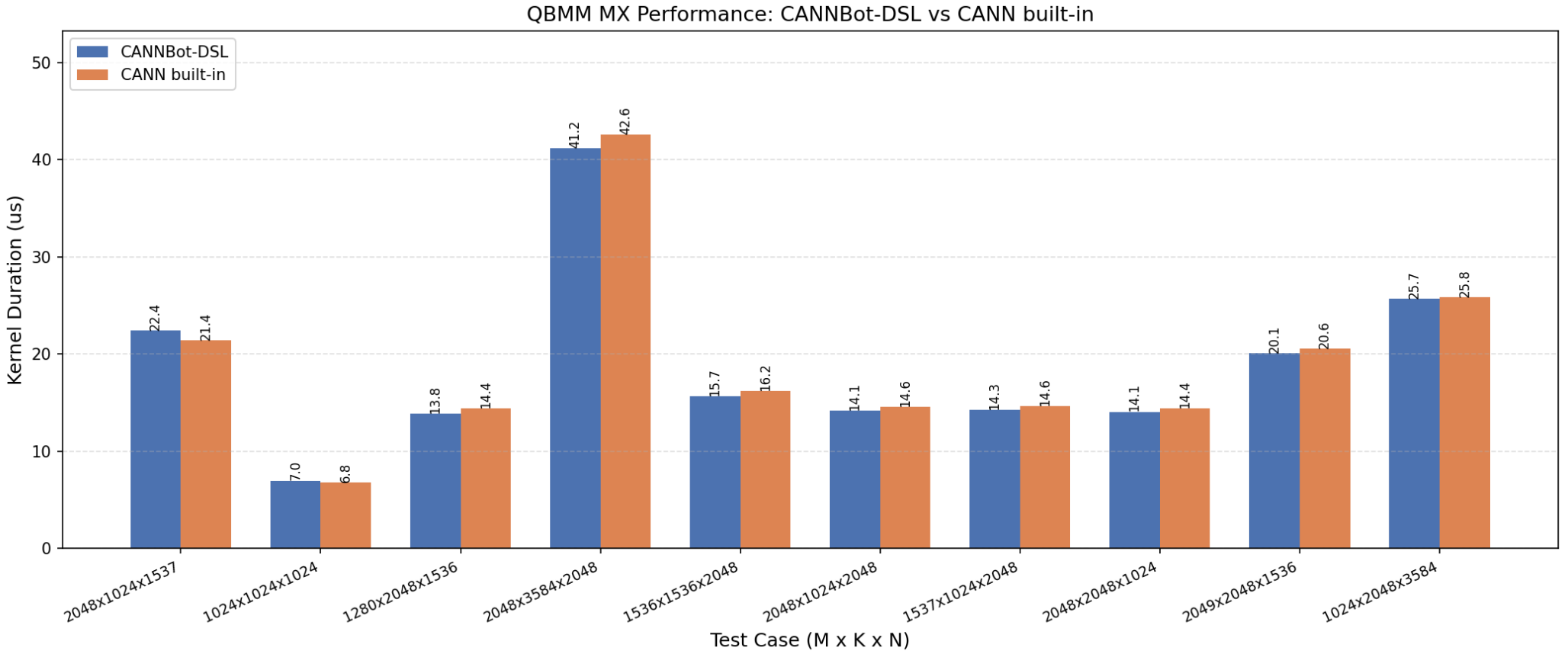
```
// 3. 通知计算完成，推进缓冲编号
SetFlag<HardEvent::M_MTE1>(flag);
l0PingPong_++;
```

表达同一数据流，减少手动维护槽位编号、地址偏移与同步事件

两侧为语义对应的节选，省略切片与累加参数。produce / consume 只选槽，同步由实际访问决定

基于CANNBot-DSL的MX 量化Matmul实践

样例完全通过与 Agent 交互迭代生成，分块、复用与流水决策清晰表达，性能结果通过实测验证



性能决策显式表达，常规资源与同步管理由编译器承接，让性能实现更容易理解、调优和迭代

性能可预期

执行结构可分析，实测结果可验证

调优有抓手

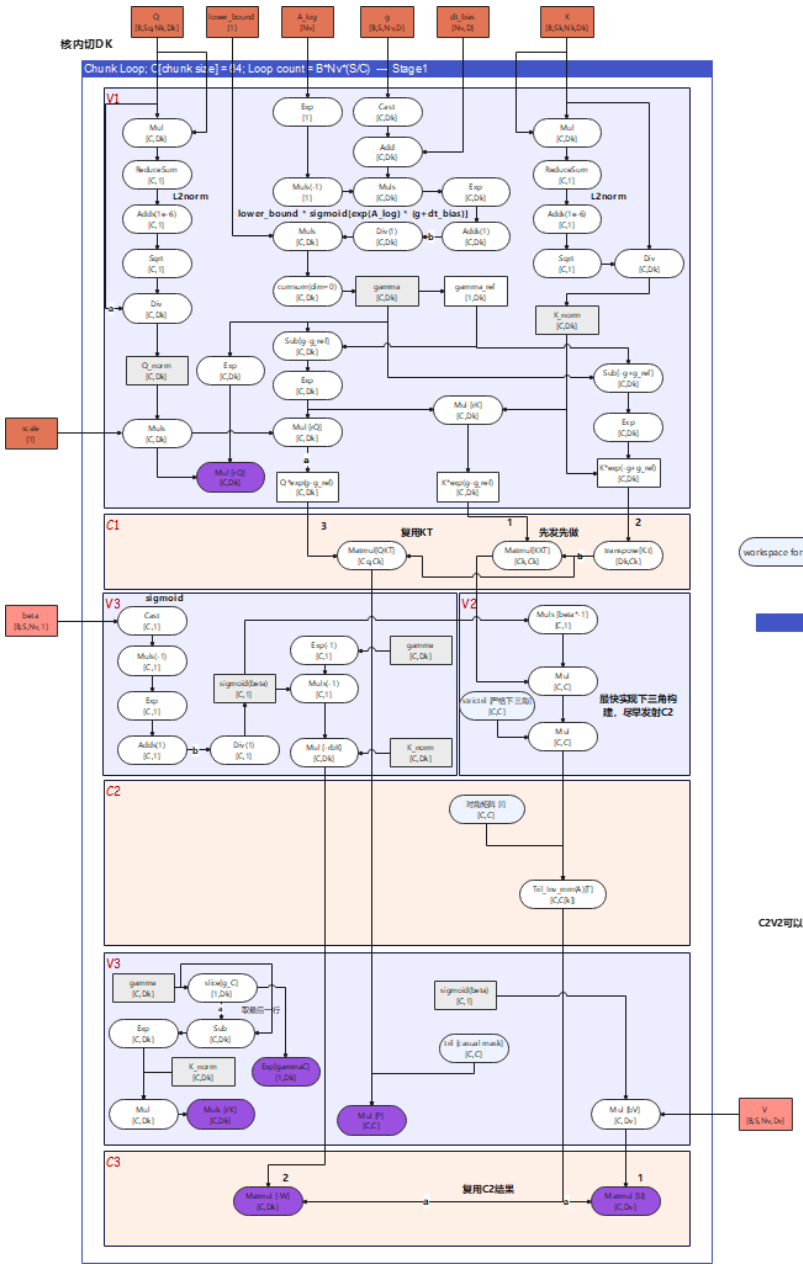
策略可定位，数据流与资源可调

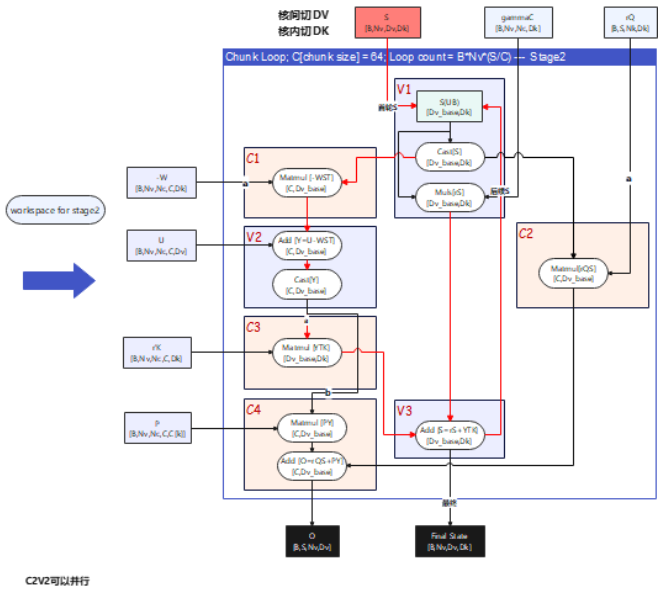
对 Agent 友好

接口与修改边界明确，便于迭代

开源样例：gitcode.com/cann/cannbot-dsl/tree/master/samples/matmul/quant_matmul

基于CANNBot-DSL的FlashKDA算子开发





$$\mathbf{M}_{[t]} = \left(\mathbf{I} + \mathrm{StrictTril}\left(\mathrm{Diag}\left(\beta_{[t]}\right)\left(\mathbf{\Gamma}_{[t]}^{1\to C} \odot \mathbf{K}_{[t]}\right)\left(\frac{\mathbf{K}_{[t]}}{\mathbf{\Gamma}_{[t]}^{1\to C}}\right)^{\top}\right)\right)^{-1} \mathrm{Diag}\left(\beta_{[t]}\right)$$

$$\mathbf{W}_{[t]} = \mathbf{M}_{[t]}\left(\mathbf{\Gamma}_{[t]}^{1\to C} \odot \mathbf{K}_{[t]}\right), \qquad \mathbf{U}_{[t]} = \mathbf{M}_{[t]}\mathbf{V}_{[t]}$$

$$\mathbf{S}_{[t+1]} = \mathrm{Diag}(\gamma_{[t]}^{C})\mathbf{S}_{[t]} + \left(\mathbf{\Gamma}_{[t]}^{i\to C} \odot \mathbf{K}_{[t]}\right)^{\top}\left(\mathbf{U}_{[t]} - \mathbf{W}_{[t]}\mathbf{S}_{[t]}\right) \in \mathbb{R}^{d_k \times d_v}$$

$$\mathbf{O}_{[t]} = \underbrace{\left(\mathbf{\Gamma}_{[t]}^{1\to C} \odot \mathbf{Q}_{[t]}\right)\mathbf{S}_{[t]}}_{\text{inter chunk}} + \underbrace{\mathrm{Tril}\left(\left(\mathbf{\Gamma}_{[t]}^{1\to C} \odot \mathbf{Q}_{[t]}\right)\left(\frac{\mathbf{K}_{[t]}}{\mathbf{\Gamma}_{[t]}^{1\to C}}\right)^{\top}\right)}_{\text{intra chunk}} \underbrace{\left(\mathbf{U}_{[t]} - \mathbf{W}_{[t]}\mathbf{S}_{[t]}\right)}_{\text{“pseudo”-value term}} \in \mathbb{R}^{C \times d_v}$$

1. 变量计算众多，Buffer分配、同步插入难

2. 矩阵求逆计算及搬运逻辑表达复杂

3. 计算阶段多导致流水调优复杂

变量计算众多，Buffer分配、同步插入难

```
self.ub_mqk_bf16 = Channel(MemLoc.UB, (32,64), dtypes.bfloat16,
                           depth=1, data_format="nd")
self.ub_bf16_nz = Channel(MemLoc.UB, (64, 64), dtypes.bfloat16,
                          depth=2, data_format="nz", n1_pad=16)
```

Buffer简单分配

Buffer分配相关的实现一行说清，不必费心计算各种偏移，方便灵活调整。

```
# ① Vector 侧：计算结束发起搬运
mem_copy(l1_dst, ub_nz_src.consume(), engine=make_copy_engine
                (split_axis=1), part_id=self.subblock_idx)

# ② 通道声明：跨核指令Buffer分配
l1_dst = Channel(MemLoc.L1, (64, 128),dtypes.bfloat16, depth=1,
                 kind=ChannelKind.CrossCore).produce()

# ③ Cube 侧：直接取用，握手已内建
l0b_write = l0b_db.produce()
mem_copy(l0b_write, l1_dst, transpose=True)
```

同步插入自动化

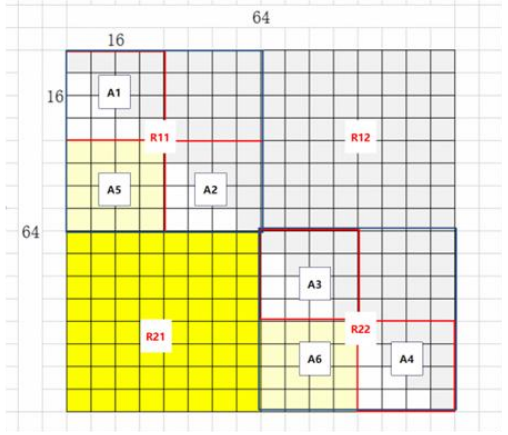
谁 生 产 、谁消费声明在 Channel 上，握手 flag 由框架精确合成。

矩阵求逆计算及搬运逻辑表达复杂

矩阵分解

纯CUBE运算+巧妙布局L0C累加

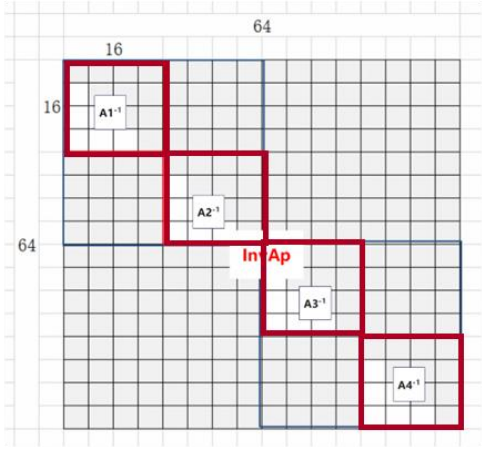
通过矩阵切分、小矩阵打包求解、L0C持久化累加中间结果加速小尺寸矩阵乘运算



STEP1

16×16小矩阵：Neumman级数倍增

$$(I-L)^{-1} = I + L + L^2 + \cdots + L^{C-1}$$

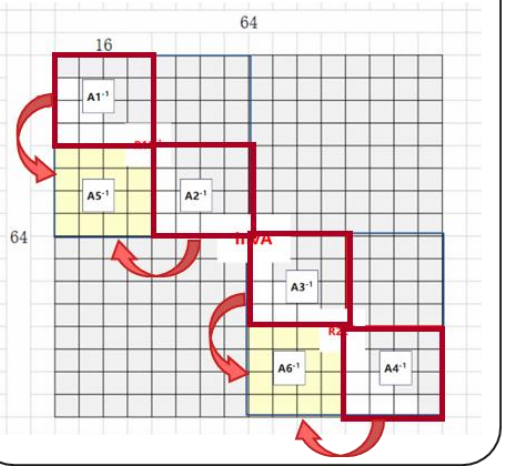


STEP2

32×32中矩阵：小矩阵分块迭代

$$INV = \begin{bmatrix} INV_{UL} & 0 \\ INV_{LL} & INV_{LR} \end{bmatrix}, \quad L = \begin{bmatrix} L_{UL} & 0 \\ L_{LL} & L_{LR} \end{bmatrix}$$

$$INV_{LL} = -INV_{LR} \times L_{LL} \times INV_{UL}$$

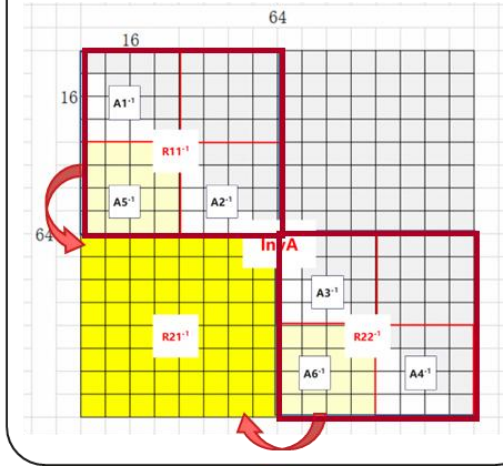


STEP3

64×64大矩阵：中矩阵迭代

$$INV = \begin{bmatrix} INV_{UL} & 0 \\ INV_{LL} & INV_{LR} \end{bmatrix}, \quad L = \begin{bmatrix} L_{UL} & 0 \\ L_{LL} & L_{LR} \end{bmatrix}$$

$$INV_{LL} = -INV_{LR} \times L_{LL} \times INV_{UL}$$



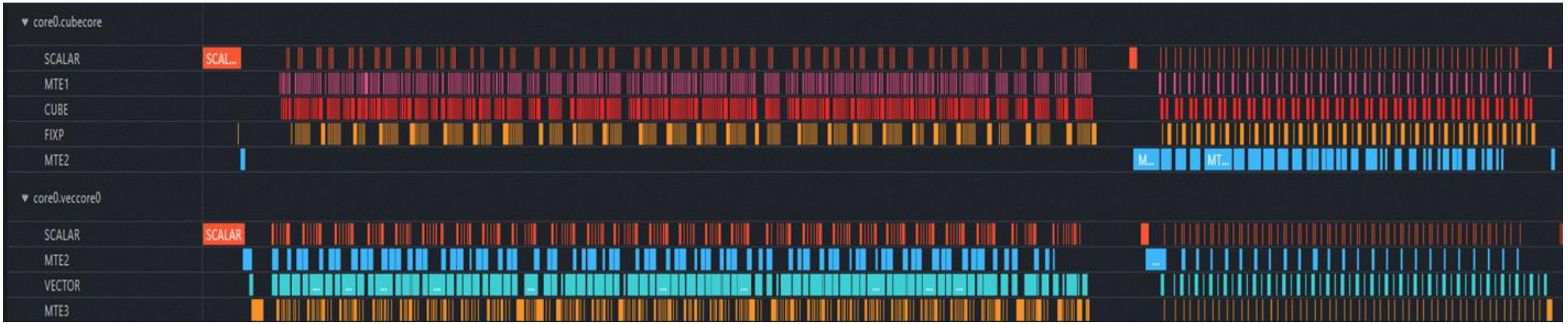

```
# 典型场景搬运：直接声明格式转换
self.fixpipe_u = make_copy_engine(split_axis=0)
self.ub2l1    = make_copy_engine(format_transform='nd2nz')

# 16×16映射：4 个对角块，源步进 / 目标步进用几何描述
mem_copy( l0a_slot, left_l1, engine=self.packed64_l12l0,)
self.packed_l12l0 = make_copy_engine(format_transform='nd2nz',
                    block_mapping=CopyBlockMapping((16, 16),
              repeats=4,src_step=(16, 16), dst_step=(16, 16)))
```

分形搬运参数化成几何映射

手写 Nd2NzParams 十几个字段（行列/步长/gap/repeat）在DSL 默认实现，特殊情况也只是简单易懂的几何映射。

计算阶段多导致流水调优困难




```
# 2 深延迟线：让 "一拍前的任务" 的元数据可被类型化地读取
task_delay_line = DelayLineGroup(2, "idx", "valid_rows")

# 发射侧：任务入队，携带本拍元数据
task_delay_line.push(idx=local_task_index,
valid_rows=current_valid_rows)
# 消费侧：tap(1) 保证取到上一拍 push 的值；奇偶代际选槽
delayed_task_index = task_delay_line.idx.tap(1)
buffer_parity = delayed_task_index % 2
...
# 消费完成进行切换
task_delay_line.advance()
```

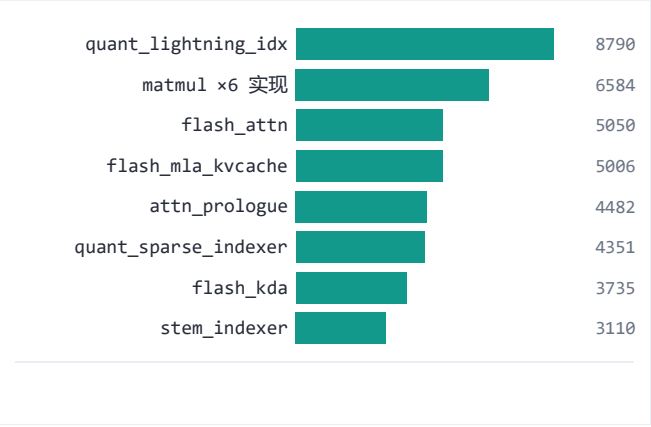
延迟是类型，不是约定

跨任务 / 跨轮 ping-pong 由框架保证迭代之间的不同计算的数据正确性。

CANNBot-DSL： Agent Friendly 高性能复杂算子开发范式

01 Agent Friendly

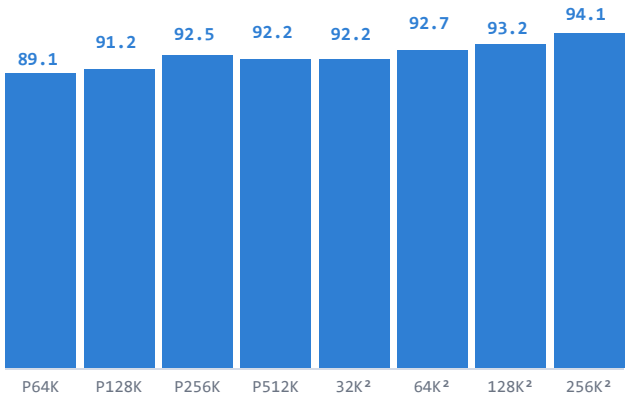
26 个算子



代码仓中有 attention / MLA / 稀疏 / 全量化这类生产级难题。代码仓中的代码全部是基于CANNBot生成和调优，无手写代码

02 高性能

90+ % MFU



纵轴 0 ～ 50% MFU ・ msprof预热 1 次计时 10 次取平均

DSV4.1 Prefill SMLA算子性能。

T=1024/2048/3072/4096 TOPK=512 89.1/91.2/92.5/92.2

T=4096 TOPK=512/1024/1536/2048 92.2/92.7/93.2/94.1

03 手写效率

3 倍

01	~~PingPong / N-buffer 多级流水轮转~~	0 行
02	~~set_flag / wait_flag 手工同步~~	0 行
03	~~Cube / Vector 拆核与跨核交接~~	0 行
04	~~分形 / NZ 私有格式地址推导~~	0 行
05	~~尾块分支与 32B 对齐补齐~~	0 行
06	~~TilingData 跨 Host / Kernel 贯通~~	0 行

写算子的几个痛点,CANNBot-DSL进行了优化。经过抽象，代码量降低为原有的1/3。

https://gitcode.com/cann/cannbot-dsl

更多CANNBot-DSL性能数据

01 engram_gate hc=4、D=5120、BF16

T	数据量 MB	TileLang μs	DSL μs
36	4.9	5.74	5.66
72	9.7	8.95	7.99
256	34.2	24.59	18.85
512	68.2	46.98	33.58
1024	136.4	92.52	72.00
2048	272.7	183.07	162.04
4096	545.3	364.11	365.04

DSL 大 shape 回落

DSL 带宽自 t=512 的 2032 单调跌到 1494 GB/s，数据量超 L2 后复用优势被摊薄。

两端持平，成因不同

小端两边带宽仅 ~850 GB/s，被固定开销支配；大端两边同时收敛到 ~1495 GB/s。

02 GQA forward BF16、D=128、无 mask

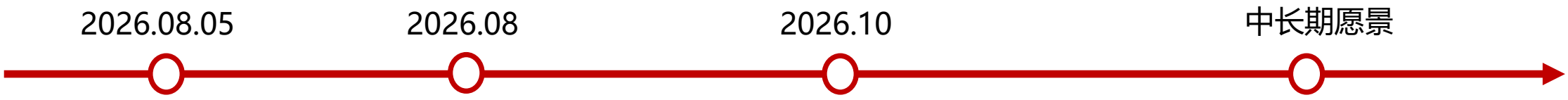
Sq	Skv	G	TileLang μs	DSL μs
256	256	8	5.46	8.56
128	512	28	7.60	10.20
512	512	8	13.37	12.74
512	4096	28	100.78	96.65
2048	2048	8	107.27	63.05
4096	8192	28	2647.28	1437.06
32K	32K	8	23651.87	13145.75

能力	CANNBot-DSL	TileLang 当前示例
输入布局	BNSD、BSND、TND；Q 与 KV 布局可分别指定	扁平二维序列：Q/O 为 [Sq×G,D]，K/V 为 [Skv,D]
头数 / GQA	支持 Nq、Nkv 参数化，覆盖 MHA/GQA	通过展平 query 表示单共享 KV head 的 GQA；MHA 可视作 G=1
数据类型 / D	D=64/128/256；支持 FP16/BF16 路径	BF16，D 固定 128
Mask / 长度	full、causal、sliding-window；支持 TND、cu_seqlens / sequsd	当前示例为无 mask；不覆盖变长序列和窗口 mask
Paged KV	支持 PA_BNBD、PA_BBND、PA_NZ 等 KV 布局	当前示例不支持 Paged KV

https://gitcode.com/cann/cannbot-dsl

Ascend950PR· CANN 9.2.0-beta.2 cannbotdsl 0.7.0 vs tilelang 0.1.15

CANNBot-DSL Roadmap



首次开源

- 发布自定义开发、测试功能，支持社区用户开发验证
- 优化基于CANNBot-DSL的算子开发skills

whl包发布

- 发布了基于CANNBot+CANNBot-DSL生成的DeepSeekV4.1和KimiK3算子样例
- 接口文档开放

典型模型复杂算子发布

- 完成基于CANNBot-DSL超复杂算子开发方法论探索
- 通过CANNBot-DSL协助客户算子商用部署

生态共建，智能演进

代码仓地址：https://gitcode.com/cann/cannbot-dsl

欢迎前往仓库查阅体验！

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann